Constraint Programming - The Estates (MiniZinc Practice)

Declarative Programming, Applications
Mr. Bermeo Quito, Wilber Eduardo
Dr. Mateu Villaret Auselle
January 16, 2026
University of Girona

Miquell Coll Barneto
u1987334
u1987334@campus.udg.edu

Ramon López i Cros
u1988987
u1988987@campus.udg.edu

Índex

1 Data Model **2**

2 Viewpoints **3**
2.1 Viewpoint 1 3
2.2 Viewpoint 2 8
2.2.1 Model Constraints 8

3 Results **10**
3.1 Input instances 10
3.2 Viewpoint 1 14
3.2.1 Conclusions 15
3.2.2 System data 16
3.3 Viewpoint 2 16
3.3.1 Comparison of solvers and heuristics 16
3.3.2 System data 17

4 Comments and reflections **17**

1 Data Model

We started the problem by addressing the data model, as it would be the foundation for both of us to work in a coordinated manner. The first thing we did was define what data the user should input and how it should be treated within the model. From this came our first skeleton of the data model (See figure1).

```
1 type Pos = tuple (int, int);
2
3 int: n_tipus_finques;
4 set of int: Tipus Finca 1..n_tipus_finques;
5
6% Especificació tipus de finques
7 array[Tipus Finca] of set of Pos: definicio_finques;
8
9% Dimensions de la parcel·la a edificar
10 int: height;
11 int: width;
12
13% Caselles destinades a parcs
14 set of Pos: parcs;
15
16% Caselles destinades a camins
17 set of Pos: camins;
18
19% Finques que s'han de posar, indicat el tipus que és 20 int: n_finques;
21 array[1..n_finques] of Tipus Finca: finques;
22
23% Parelles de finques que no es poden tocar
24 type Incompatibles = tuple (Tipus Finca, Tipus_Finca);
25 set of Incompatibles: incompatibles;
26
27% Finques que han de tenir com a veí una casella de parc com a minim
28 set of int: Index Finca = 1..n_finques;
29 set of Index Finca: finques amb parc;
```

Figura 1: Initial skeleton of the data model

In this initial design, things were quite intuitive: The board: Simply two integers to define the height and width. Parks and paths: We defined them as SETs of coordinates (row-column tuples). This was the fastest way to mark already occupied cells. The estates to be placed: An array where each position indicated the type of estate (from 1 to 13) to be placed. Neighborhood restrictions: For estates that cannot touch, we thought of a set of pairs (tuples with the estate IDs). Estates with park: For estates needing a park nearby, we defined a set with their indices.

This model was enough to start, but when we each began thinking about our own ViewPoint, we realized a problem: rotations. Encoding the turns and flips of each piece was a mess. Here we came up with two different solutions (explained later), so we adapted the skeleton data model to each ViewPoint. Thus, we ended up with two ViewPoints that are almost identical regarding input data, but differ in the definition of estate types (See figures 2 and 3).

2 Viewpoints

2.1 Viewpoint 1

When defining the ViewPoint, I decided that the best way to represent the problem was to focus on the estate as a whole object instead of seeing it as a set of cells. This way, the solver would be abstracted from the cohesion between cells within the same estate. Therefore, my model is based on three variables for each estate on the board: its position (row and column) and its specific orientation (some estates have 8 orientations, others have fewer). This approach allows the solver not to have to "discover"the shape of the estate at each step, but simply to place one of the orientations of the estate.

Regarding orientations, instead of having a single static shape, each estate is defined by a set of possibilities that include 90, 180, and 270-degree rotations, as well as their symmetries[1]. This rotation logic is the key to this model, as it allows an estate to fit according to how it is rotated, always respecting its original shape.

Now, to organize all the information so that the program can read it quickly, I have structured all orientations in a large three-dimensional *array*. To know which orientations correspond to each estate, a traceability system based on two auxiliary *arrays* is used; one indicating where the block of orientations for each estate type begins and another telling us how many orientations it has in total. This way, when the model needs to place, for example, estate type 5, the ViewPoint has a *constraint* so that it can only choose from the n orientations defined for that piece.

Although decisions are made about the estate's starting position and its orientation, the board is responsible for verifying that estates do not overlap, that no cell is outside the board, or that parks and paths are not overridden by estates. Thus, the solver acts as follows: it first decides where an estate goes and which rotation is placed, and then lets the board constraints determine if the configuration is valid.

[1] Not all estates have symmetries or rotations

```
include "global_cardinality.mzn";

% ----------------------------- %
% -----  INPUT DATA  ---------- %
% ----------------------------- %
int: board_height;
int: board_width;

int: n_estates;
array[1..n_estates] of int: estates;

int: n_parks;
array[1..n_parks, 1..2] of int: parks;

int: n_paths;
array[1..n_paths, 1..2] of int: paths;

int: n_incompatibles;
array[1..n_incompatibles, 1..2] of 1..n_estates: incompatibles;

set of int: estates_with_park;

% ----------------------------- %
% ---  CONFIGURATION DATA  ---- %
% ----------------------------- %

int: n_types;

int: total_orientations;
int: max_cells;

array[1..n_types] of int: start_orientations;
array[1..n_types] of int: orientations_per_estate;
array[1..n_types] of int: cells_per_estate;

array[1..total_orientations, 1..max_cells, 1..2] of int: estates_with_orientations;

% ----------------------------- %
% ----  DECISION VARIABLES ---- %
% ----------------------------- %

array[1..board_height, 1..board_width] of var 0..n_estates: board; % Board

array[1..n_estates] of var 1..board_height: row; % Row of the starting cell of each estate
array[1..n_estates] of var 1..board_width: col; % Column of the starting cell of each estate
array[1..n_estates] of var 1..total_orientations: orientation_id; % Orientation of each estate

% ----------------------------- %
% --------  CONSTRAINTS ------- %
% ----------------------------- %

% How does it work?
% - It goes through the estates to be placed, and tries to place them WHOLE,
%     i.e., in a cell C of the board must go cell F of the estate
%     and check that all other cells do not break any restrictions so we can place them.

% Place estates
constraint forall(f in 1..n_estates) (
    let {
        int: estate = estates[f], % Estate
        var int: orientation = orientation_id[f] % Orientation of the estate
    } in (
```

```
        orientation >= start_orientations[estate] /\  % The orientation (from the array of all orientations) belongs to the estate's range (>= first)
        orientation < start_orientations[estate] + orientations_per_estate[estate] /\ % ...up to the number of orientations this estate has

        forall(cell in 1..cells_per_estate[estate]) ( % Check that the cells can be placed on the board
            let {
                var int: cell_row = row[f] + estates_with_orientations[orientation, cell, 1] - 1, % Get the cell row starting from the current row + the model offset
                var int: cell_col = col[f] + estates_with_orientations[orientation, cell, 2] - 1 % Same with column
            } in (
                cell_row in 1..board_height /\ cell_col in 1..board_width /\ % The appropriate cell is within the board
                board[cell_row, cell_col] = f % Put the estate there
            )
        )
    )
);

int: paths_or_parks = (board_height * board_width) - sum(f in 1..n_estates)(cells_per_estate[estates[f]]); % These are the ones that should have park or path

% No overlapping cells (Global Cardinality)
constraint global_cardinality(
    [board[_row, _col] | _row in 1..board_height, _col in 1..board_width], % Each cell of the board
    [_estate | _estate in 0..n_estates], % 0(paths or parks) ++ Estate values (1, 2, 3, 4, ...) -> What we control
    [paths_or_parks] ++ [cells_per_estate[estates[_estate]] | _estate in 1..n_estates] % paths_or_parks + cells_per_estate[estates[_estate]] for each estate
);

% Extra to optimize a bit (negligible) Avoid duality between equal estates
constraint forall(estate1, estate2 in 1..n_estates where estate1 < estate2) ( % For each pair of estates
    if estates[estate1] == estates[estate2] then % The same estate twice
         % Eliminate the possibility of counting 2 solutions with the estates swapped (it would be the same)
        (row[estate1] * board_width + col[estate1]) < (row[estate2] * board_width + col[estate2])
    endif
);

% Parks (cannot have estate)
constraint forall(park in 1..n_parks) (
    board[parks[park, 1], parks[park, 2]] = 0  % board[row, column] = 0
);

% Paths (cannot have estate)
constraint forall(path in 1..n_paths) (
    board[paths[path, 1], paths[path, 2]] = 0 % board[row, column] = 0
);

% Estates that cannot touch
constraint forall(i in 1..n_incompatibles) ( % For each pair of incompatibles (each pair is a row)
    let {
        int: estate1 = incompatibles[i, 1],
        int: estate2 = incompatibles[i, 2]
```

```
    } in
    forall(_row in 1..board_height, _col in 1..board_width where board[_row,_col] == estate1) ( % For each cell (row, col) of the estate on the board
        forall(_offset_row, _offset_col in -1..1 where (_offset_row != 0 \/ _offset_col != 0) /\ not (_offset_row != 0 /\ _offset_col != 0)) ( % For each cell at distance 1 (neighbor) except the cell itself (when both are 0)

            let {
                int: _new_row = _row + _offset_row,
                int: _new_col = _col + _offset_col
            } in (
                if _new_row in 1..board_height /\ _new_col in 1..board_width then % must belong to the board
                    board[_new_row, _new_col] != estate2 % Here we check different
                endif
            )
        )
    )
);

% Internal boolean matrix for parks
array[1..board_height, 1..board_width] of bool: park_matrix = array2d(1..board_height, 1..board_width, [
    exists(_park in 1..n_parks)(parks[_park,1] == _row /\ parks[_park,2] == _col) % There exists a park with this position
    | _row in 1..board_height, _col in 1..board_width
]);

% Estates with park
constraint forall(_estate in estates_with_park) (
    exists(_row in 1..board_height, _col in 1..board_width where board[_row, _col] == _estate) ( % Any of the cells of the estate
        exists(_offset_row, _offset_col in -1..1 where (_offset_row != 0 \/ _offset_col != 0) /\ not (_offset_row != 0 /\ _offset_col != 0)) ( % Any cell at distance 1 (neighbor) except the cell itself (when both are 0)

            let {
                int: _new_row = _row + _offset_row,
                int: _new_col = _col + _offset_col
            } in (
                _new_row in 1..board_height /\ _new_col in 1..board_width /\ park_matrix[_new_row, _new_col] % Must be within the board and be a park
            )
        )
    )
);

% ----------------------------- %
% ---------  SOLVE  ----------- %
% ----------------------------- %
solve :: int_search(row ++ col ++ orientation_id, first_fail, indomain_min) satisfy; % Search prioritizing (first_fail) oriented to estates, not cells (row ++ col ++ orientation_id). indomain_min tries the smallest value in the domain first

% ----------------------------- %
% --------  OUTPUT  ----------- %
% ----------------------------- %
output [
    if col_iter = 1 then "\n" else "" endif ++
    let {
        int: value = fix(board[row_iter, col_iter]),
```

```
        bool: p = exists(k in 1..n_parks)(parks[k,1]==row_iter /\ parks[k,2]==col_iter),
        bool: c = exists(k in 1..n_paths)(paths[k,1]==row_iter /\ paths[k,2]==col_iter)
    } in (
        if p then "Park  "
        elseif c then "Path  "
        elseif value == 0 then ".     "
        elseif value < 10 then show(value) ++ "     "
        else show(value) ++ "    " endif
    )
    | row_iter in 1..board_height, col_iter in 1..board_width
] ++ ["\n"];
```

2.2 Viewpoint 2

The viewpoint I chose to model the problem consists of explicitly representing the *position of each cell that is part of the estates* within the overall plot. More specifically, each estate is decomposed into a set of elementary points, and each of these points is modeled as a variable indicating its linear position within the grid.

Thus, if the total number of squares forming all estates is N, the model uses an array of variables:

$$\texttt{solution}[1..N]$$

where each variable takes values in the domain $\{1, \ldots, \texttt{height} \times \texttt{width}\}$, which corresponds to the possible positions within the plot, encoded in linear format. This viewpoint allows geometric, adjacency, and non-overlap constraints to be expressed naturally through arithmetic operations on coordinates.

2.2.1 Model Constraints

The main constraints ensuring the correctness of the obtained solutions are described below.

Non-overlap of estates The global constraint `alldifferent(solution)` ensures that no two squares occupied by estates share the same position within the plot. This constraint guarantees that estates do not overlap.

```
constraint alldifferent(solution);
```

Correct placement of estate shapes Each estate has an associated relative geometric definition, specified using relative coordinates between its points. The model imposes that, for any pair of points within the same estate, the relative distance between their positions on the grid matches one of the allowed distances according to its definition, considering rotations and symmetries.

```
array[Index_Estate] of int: start = [ 1 + sum(t in 1..k-1)(npts[t]) | k in
↪  Index_Estate ];
function set of int: idxs(int: k) = start[k] .. start[k] + npts[k] - 1;

type D = tuple(int,int);

function var D: distance(var int: a, var int: b) =
  ( abs(linearToRow(a) - linearToRow(b)),
    abs(linearToCol(a) - linearToCol(b)) );


function array[1..2] of D: possible_distances(int: k, int: i, int: j) =
  let {
    int: t = estates[k],
    Pos: a = estate_definition[t, i],
    Pos: b = estate_definition[t, j],
    int: dr = abs(a.1 - b.1),
    int: dc = abs(a.2 - b.2)
  } in
    [ (dr, dc), (dc, dr) ];
```

```
function int: local_idx(int: k, int: g) = g - start[k] + 1;

constraint forall(k in Index_Estate)(
  forall(i, j in idxs(k) where i < j)(
    let {
      int: li = local_idx(k,i),
      int: lj = local_idx(k,j),
      var D: d = distance(solution[i], solution[j]),
      array[1..2] of D: P = possible_distances(k, li, lj)
    } in
      (d == P[1]) \/ (d == P[2])
  )
);

```

This constraint ensures that each estate appears on the plot in a valid shape consistent with its type.

Prohibition of occupying parks and paths It is imposed that no variable in `solution` takes a value corresponding to a cell designated as a park or path, thus preventing estates from being built there.

```
set of int: forb = parks union paths;
constraint forall(p in 1..N)(not (solution[p] in forb) );
```

Incompatibility constraints between estates Some pairs of estates cannot be neighbors. For these pairs, the model imposes that no point of one estate is adjacent to any point of the other estate. This constraint is expressed by checking all pairs of involved points.

```
function var bool: adjacent4(var int: a, var int: b) =
  abs(linearToRow(a) - linearToRow(b)) + abs(linearToCol(a) - linearToCol(b)) ==
  ↪ 1;

constraint forall(m in 1..n_incompatibles)(
  let {
    int: k1 = incompatibles[m].1,
    int: k2 = incompatibles[m].2
  } in
  forall(p1 in idxs(k1), p2 in idxs(k2))(
    not adjacent4(solution[p1], solution[p2])
  )
);
```

Estates with adjacent park requirement For certain estates, it is required that at least one of their squares be adjacent to a park cell. The model expresses this condition using an existential constraint that ensures the existence of a point of the estate adjacent to some park position.

```
constraint forall(k in estates_with_park)(
  exists(p in idxs(k), q in parks)(
    adjacent4(solution[p], q)
  )
);
```

3 Results

3.1 Input instances

To generate the results, we ran our models with 10 instances of varying difficulty and with 2 different solvers (and different heuristics). All this to see exactly how the solvers work and the differences between the possible heuristics.

Below, the different instances used are listed (paths in red and parks in yellow. The relationships are detailed with each figure):

Figura 2: First instance

Estates: [3, 5, 8, 13] Parks: [ ] Paths: [ ] Incompatibilities: [ ] Estates with park: [ ]

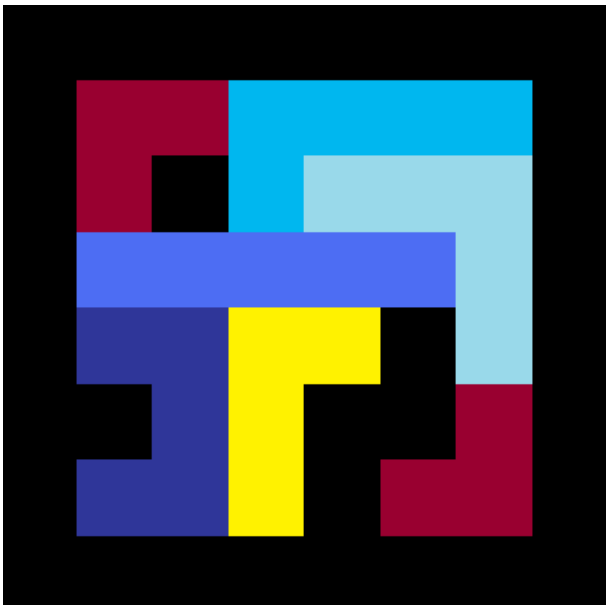

Figura 3: Second instance

Estates: [2, 3, 7, 8] Parks: [(1,1)(1,2)(2,1)(5,6)(6,5)(6,6)] Paths: [(4,3)(4,4)(5,3)(6,3)] Incompatibilities: [ ] Estates with park: [ ]

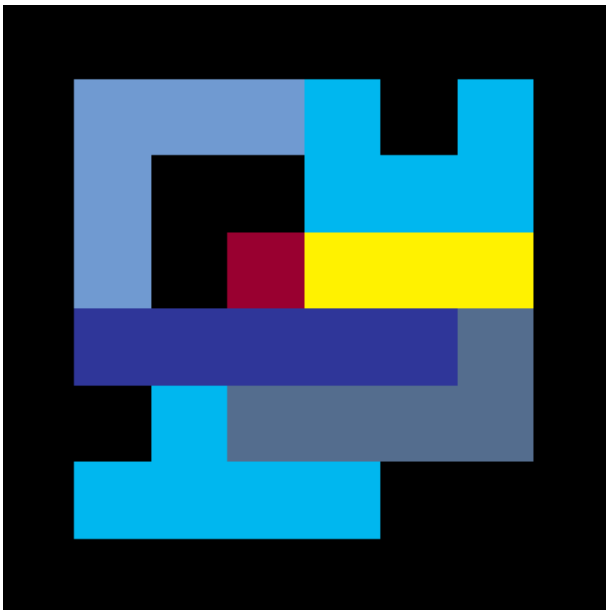

Figura 4: Third instance

Estates: [2, 3, 7, 8, 11] Parks: [(3, 3)] Paths: [(3, 4)(3, 5)(3, 6)] Incompatibilities: [ ] Estates with park: [1]

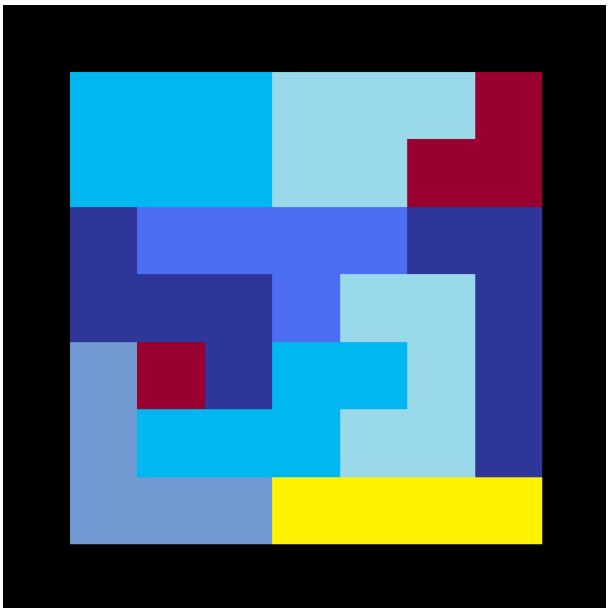

Figura 5: Fourth instance

Estates: [3, 4, 5, 7, 8, 11, 12, 13] Parks: [(1,7)(2,6)(2,7)(5,2)] Paths: [(7,4)(7,5)(7,6)(7,7)] Incompatibilities: [(1, 7)] Estates with park: [5]

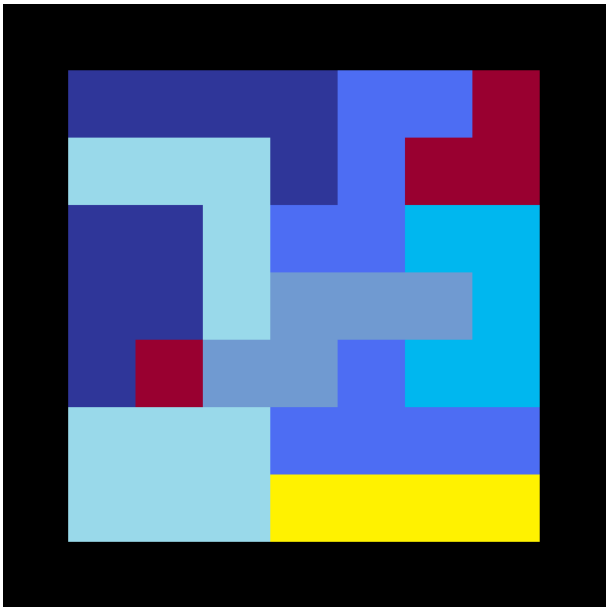

Figura 6: Fifth instance

Estates: [3, 4, 5, 7, 8, 11, 12, 13] Parks: [(1,7)(2,6)(2,7)(5,2)] Paths: [(7,4)(7,5)(7,6)(7,7)] Incompatibilities: [(1, 8)] Estates with park: [4]

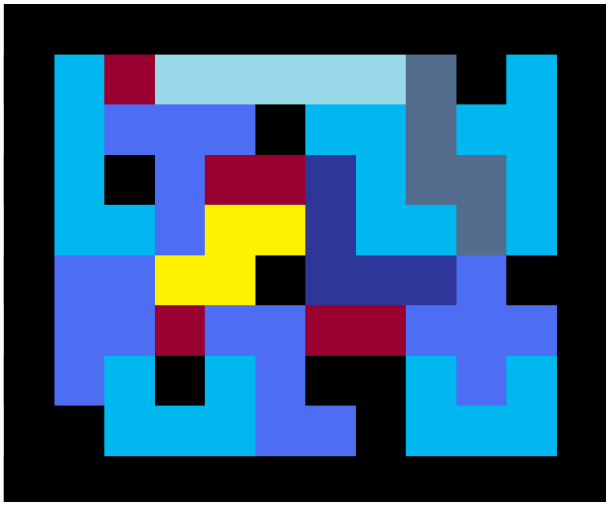

Figura 7: Sixth instance

Estates: [2,3,4,5,6,7,7, 8,10,11,12, 12] Parks: [(3,4)(3,5)(6,3)(6,6)(6,7)(1,2)] Paths: [(4,4)(4,5)(5,3)(5,4)] Incompatibilities: [(1,8)(2,8)(3,7)] Estates with park: [1, 5, 9]

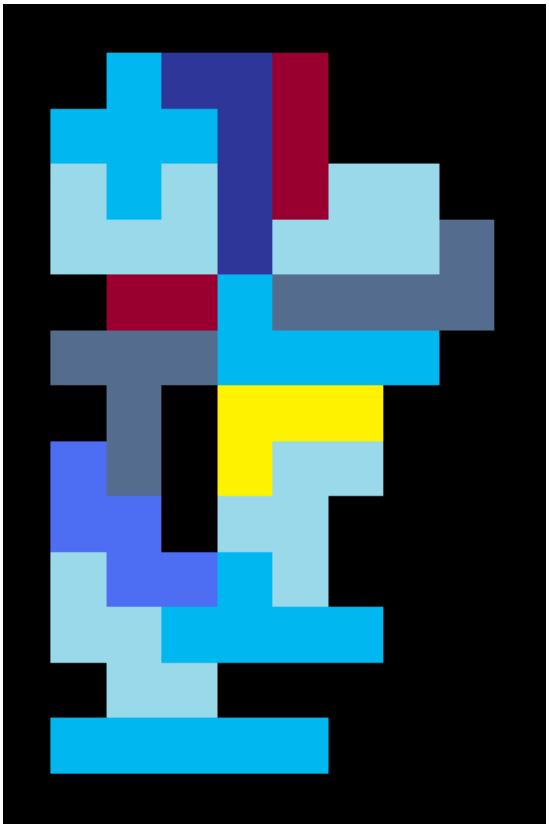

Figura 8: Seventh instance

Estates: [1, 2, 3, 3, 3, 5, 6, 7, 9, 9, 10, 11] Parks: [(1,5)(2,5)(3,5)(5,2)(5,3)] Paths: [(7,4)(7,5)(7,6)(8,4)] Incompatibilities: [(1,7)(1,8)(1,9)(1,10)] Estates with park: [5, 6]

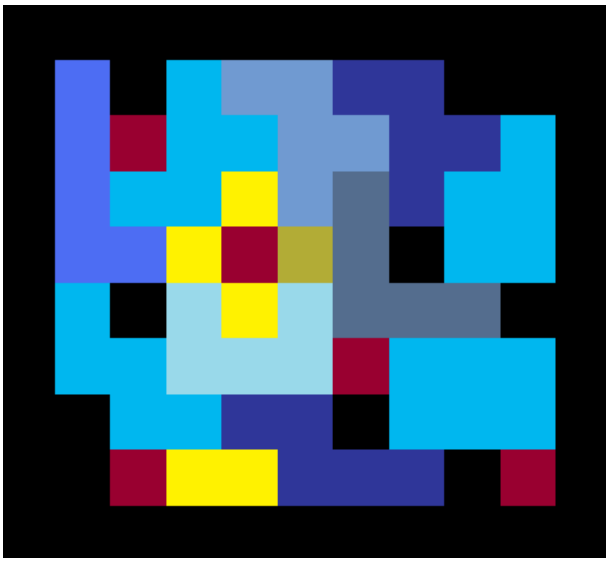

Figura 9: Eighth instance

Estates: [1, 1, 1, 3, 4, 5, 7, 8, 9, 13] Parks: [(2,2)(4,4)(6,6)(8,2)(8,9)] Paths: [(3,4)(4,3)(4,5)(5,4)(8,3)(8,4) Incompatibilities: [(1,6)(3,4)(9,10)(4,7)] Estates with park: [1,7]

Figura 10: Ninth instance (UNSAT)

Estates: [5, 5, 6, 7, 8, 9, 10, 11, 13] Parks: [(1,7)(1,8)(2,7)(2,8)(3,7)(3,8)(4,7)] Paths: [(5,8)(6,1)(6,2)(7,1)(7,2)(7,3)(8,2)] Incompatibilities: [(1,9)(2,9)(3,7)(4,5)(6,8)(1,4)] Estates with park: [3, 5, 9]

Figura 11: Tenth instance (UNSAT)

Estates: [1,2,3,4,5,6,7,8,9,10,11,12,13] Parks: [(2,2)(2,8)(5,5)(8,2)(8,8)] Paths: [(1,4)(1,6)(2,4)(2,6)(3,4)(Incompatibilities: [(1,13)(2,12)(3,11)(4,10)(5,9)(6,8)] Estates with park: [3, 6, 9, 12]

3.2 Viewpoint 1

Instance	**Normal (No search)**	**First Fail**	**Input Order**
1 (Easy)	T.O.[2]	0.144s	0.177s
2	T.O.	0.200s	0.188s
3	T.O.	0.197s	0.297s
4	T.O.	134s	312s
5	T.O.	158s	20.35s
6	T.O.	65s	11.799s
7	T.O.	72s	4.42s
8	T.O.	642s	1.2s
9 (Hard)	T.O.	(UNSAT) 125s	(UNSAT) 125s
10 (UNSAT)	(UNSAT)	(UNSAT)	(UNSAT)

Taula 1: Resolution times with Gecode 6.3.0.

[2]T.O. (Time Out): The 15-minute limit was exceeded without finding a solution.

Instance	Normal (No search)	First Fail	Input Order
1 (Easy)	0.150	0.155s	0.260s
2	0.199	0.206s	0.232s
3	0.190	2.14s	0.183s
4	3.941	47.932s	24.779s
5	7.515s	45.598s	19.463s
6	3.177s	0.624s	12.310s
7	53.24s	2.3s	4.358s
8	0.265s	0.313s	1.138s
9 (Hard)	(UNSAT)	(UNSAT)	(UNSAT)
10 (UNSAT)	(UNSAT)	(UNSAT)	(UNSAT)

Taula 2: Resolution times with Chuffed 0.13.2

3.2.1 Conclusions

With Chuffed, the analysis of the results shows that the model's performance does not follow a linear pattern (as might be expected with increasing difficulty), but rather depends on how each heuristic manages the position and orientation of the estates. The *first_fail* heuristic is not stable, showing high time peaks because, by choosing the variable with the smallest domain, it often prioritizes deciding the orientation (with domains from 1 to 8 (sometimes 1)) before the absolute position (1 to n), causing costly *backtracking* if the chosen rotation does not fit geometrically in the available space (It is much better to decide a general order and then an orientation for the piece, rather than fixing the orientation as the starting point of the problem).

In contrast, *input_ order* is more stable in more complex problems, since if the estates have been entered in order of size or difficulty (relative to each problem), it acts as a heuristic that places the most complicated piece first, gradually reducing the problem's load. Regarding the normal mode (without heuristic), Chuffed demonstrates its effectiveness in learning on the fly and not repeating past mistakes.

With Gecode, defining a search strategy is absolutely essential. The solver times out in the first instances when run in "Normal"mode ("solve satisfy", without heuristic), demonstrating that without a heuristic, Gecode tries to decide the value of each cell on the board sequentially, and since the board has *rowsxcolumns* cells, the search space is immense, and the solver gets lost.

Regarding its heuristics, it is clearly seen that *input_ order* drastically outperforms *first_fail* in the most difficult problems (like instance 8, where we go from 642s to just 1.2s). This occurs because Gecode does not learn from clauses (as Chuffed does), so if *first_fail* makes the mistake of fixing an incorrect orientation, Gecode is forced to explore that entire branch in vain until the end. In contrast, *input_ order* allows following the placement order according to the input data (placing the most complicated ones first).

Finally, and less importantly, the speed in detecting unsatisfiable problems (the last one) confirms that the use of `global_cardinality` and the initial board constraints allow the solver to discard impossible problems at the start to avoid traversing the entire search space.

3.2.2 System data

The tests were carried out on a Windows 11 and on a Victus16d with an Intel Core i7-12700H, 16GB of RAM at 4800MTUS, and a 3050ti laptop. Not executed with threads.

3.3 Viewpoint 2

	Geocode				Chuffed		
	baseline	ff-min	io-min		baseline	ff-min	io-min
d1	0,206	0,184	0,185		0,201	0,189	0,189
d2	1,872	0,345	0,337		0,297	0,244	0,334
d3	13,138	0,296	0,293		0,415	0,26	0,292
d4	205	TIMEOUT (600)	TIMEOUT (600)		TIMEOUT (600)	418	240
d5	209	TIMEOUT (600)	TIMEOUT (600)		TIMEOUT (600)	101	81
d6	TIMEOUT (600)	27,581	TIMEOUT (600)		TIMEOUT (600)	0,925	28,258
d7	3,683	0,885	0,813		3,421	1,144	1,187
d8	0,937	2,694	2,818		240	0,904	0,896
d9	TIMEOUT (600)	TIMEOUT (600)	TIMEOUT (600)		TIMEOUT (600)	UNSAT 1,47	TIMEOUT (600)
d10	TIMEOUT (600)	TIMEOUT (600)	TIMEOUT (600)		TIMEOUT (600)	UNSAT 120	TIMEOUT (600)

Figura 12: Resolution table

3.3.1 Comparison of solvers and heuristics

This section compares the results obtained using two Constraint Programming solvers (`Gecode` and `Chuffed`) combined with three search strategies: the solver's default search (Baseline), the *First-Fail with minimum value* heuristic (FF-Min), and a search with *Input Order with minimum value* (IO-Min). A time limit of 600 seconds per instance was established.

General results For the simplest instances (d1 and d2), all solver and heuristic combinations manage to find a solution in a short time. However, it can already be observed that the use of explicit heuristics (especially FF-Min) slightly improves times compared to the default search.

As the difficulty of the instances increases (d3, d4 and beyond), the differences between solvers and heuristics become much more evident. In general, the Baseline search with Gecode degrades rapidly, reaching very high times or timing out in several instances (d4, d5, d6, d9, and d10).

Impact of heuristics The FF-Min heuristic shows a very positive impact in many cases, especially with the Chuffed solver. For example, in instance d6, Chuffed with FF-Min solves the problem in less than a second, while the rest of the configurations either time out or are much slower. Similarly, in difficult instances like d4 and d5, FF-Min with Chuffed allows finding a solution within the time limit, while Gecode does not.

In contrast, the IO-Min heuristic (selecting variables in input order and assigning the minimum value) does not show a clear and consistent improvement over Baseline. In some medium instances (e.g., d2 and d3) it does reduce time, but in difficult instances it often does not avoid timeout and can be less effective than FF-Min.

Comparison between solvers In general, the results show that `Chuffed` is more robust than `Gecode` in difficult instances, especially when combined with appropriate heuristics. Chuffed manages to solve instances that Gecode cannot, or does so in significantly less time. This is particularly notable in highly constrained instances, where the combination of adjacency constraints, incompatibilities, and limited space causes a high degree of conflict.

It is also relevant to observe that Chuffed detects some cases as `UNSAT` (d9 and d10 with FF-Min) within the time limit, while other configurations are unable to conclude this.

Conclusion Based on the experiments carried out, it can be concluded that the use of search heuristics is essential to achieve acceptable performance in medium and difficult instances. Among the options tested, the combination of `Chuffed` with the FF-Min heuristic offers the best overall results, both in resolution time and in the ability to solve highly constrained instances within the established time limit.

3.3.2 System data

The tests were carried out on Ubuntu and on a Lenovo ThinkPad T14 Gen 5 with a Ryzen 7 PRO 8840U. Not executed with threads.

4 Comments and reflections

In this practical work, we have verified that, in Constraint Programming problems, difficulty depends not only on the size of the grid or the number of estates, but above all on the *formulation* (viewpoint) and the *search strategy*. Two models representing the same problem can have radically different behaviors depending on how the variables are defined and how the solver is guided during exploration.

Main difficulties The first difficulty was deciding how to handle the rotations and symmetries of the estates. In the initial data model, describing these transformations cleanly and generally was not trivial. This led us to explore two viewpoints: one that works with the estate as an object (position and orientation), and another that works with the individual cells that form all the estates. Having two approaches allowed us to compare them.

Findings on viewpoints Viewpoint 1 (estate as an object with position and orientation) is very expressive for capturing rotations and allows the solver to "decideïn higher-level terms (where to place each estate and with which orientation). However, we have observed that it is sensitive to the decision order: if the solver fixes a poorly chosen orientation too early, it can explore very costly branches before detecting the contradiction.

Viewpoint 2 (position of each cell of all estates) is more direct for expressing geometric, adjacency, and prohibition constraints (parks/paths). However, it increases the number of variables and reinforces the importance of using heuristics, especially in difficult instances.

Final conclusion In conclusion, the main takeaway is that in CP, it is not enough to "have the right constraints": they must be modeled so that propagation is effective, and the search must be guided so that the solver discovers contradictions early.